

ML Specification Report — playground-series-s3e11

Media Campaign Cost Prediction • our from-scratch agent (GBDT pool + tree-search v3)

Figures grounded in the competition's `facts.json` , `eda_summary.json` , `STATUS.md` , and `experiments_tree_v3.json` .

Contents	
Media Campaign Cost Prediction • our from-scratch agent (GBDT pool + tree-search v3)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	3
Inference procedures	4
Evaluation & Benchmarking	4
Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08	5

Overview

The task is to predict the cost of a store media campaign for 240,224 test rows (metric: RMSLE, minimized). We solve it with a from-scratch pipeline — a pool of gradient-boosted trees (CatBoost / LightGBM) refined by an Optuna fold-proxy tune, seed-bagging, and a 24-node tree search that ends on an 8-member blend. Our committed champion scores **CV OOF RMSLE 0.29565**; the NVIDIA reproduce-agent, distilling the essence of a public XGB/CatBoost ensemble kernel, reaches 0.2956 — a **dead heat between those two** (the numbers are near-identical). The tree-search v3 run later pushed the OOF mark to 0.295284 at node #22. The third benchmark agent is ahead of both: the frozen three-way score table (`benchmark_results/three_way_scores.csv`) records `aide_local` **0.2941** against our `mine_local` 0.29528, with `local_winner` set to **aide**.

Why it matters. Forecasting campaign cost helps retailers budget promotions and allocate marketing spend efficiently across stores and periods.

Data

Purpose of Data. Predict the media-campaign `cost` for each store record — a continuous-target **regression** Playground Series episode scored by RMSLE. **Data Format** is clean **tabular CSV**, entirely numeric. **Data Volume** is **360,336 training rows / 240,224 test rows**, with **15 raw predictive features** once `id` and the `cost` target are removed (all 15 are numeric, though many are low-cardinality "categorical-like" flags).

Data Quality is high: **no missing values** in either split, **0 duplicate rows**, and no covariate shift — every column's train↔test mean differs by **< 0.7%**. The one structural wrinkle is a near-perfect collinear pair, `salad_bar` ↔ `prepared_food` (**$r = 0.9998$**). The defining property of this dataset, however, is that it is **extremely low-signal**: the strongest EDA findings were:

- **Every one of the 15 raw features has $|r| \leq 0.11$ with `cost`**. The strongest single correlations are `florist` (−0.110), `video_store` (−0.107), `prepared_food / salad_bar` (−0.099), and `store_sqft` (−0.049 Pearson / −0.056 Spearman).
- **The target is near-symmetric, not right-skewed** — `cost` $\in [50.79, 149.75]$, mean 99.61, skew 0.019, kurtosis −1.26; after `log1p` the skew is ≈ -0.34 . (We still model `log1p(cost)` because RMSLE = RMSE on `log1p`.)
- **`store_sqft` (20 distinct values) crossed with the 5 amenity flags** (`coffee_bar`, `video_store`, `salad_bar`, `prepared_food`, `florist`) forms **~111 "store profiles"**, each repeated thousands of times; a profile-mean predictor alone gives in-sample **$R^2 \approx 0.06$** — far above any single feature, making the store profile the strongest signal (usable only via out-of-fold encoding to avoid leakage).
- The **test set contains 19 store-profile combos unseen in train**, so the target encoding needs a global-mean fallback.

Annotation Guidelines. The label is `cost`, a continuous float (328 unique values in train). Submissions are **predicted campaign costs** scored by RMSLE — no thresholding or class decision is involved.

Feature Set. From the 15 raw columns we engineer **21 features**, grouped below:

Group	Features
Raw	the 15 original numeric columns
Counts / ratios	<code>amenity_count</code> , <code>weight_per_case</code> (<code>gross_weight/units_per_case</code>), <code>sales_ratio</code> (<code>store_sales/unit_sales</code>), <code>children_away</code> , <code>cars_per_child</code>
Encoded	<code>store_te</code> — K-fold target encoding of the store-profile combo (fold-internal fit; test = mean of 5 folds; global-mean fallback for unseen combos)

Splitting strategy. A single canonical **5-Fold KFold (shuffle, seed = 42)** — there is no time or group structure, and each store profile repeats thousands of times so a random split is safe; per-fold RMSLE is stable (0.2947–0.2976), confirming CV trustworthiness. The critical safeguard is **fold-safe target encoding** — `store_te` is fit only on the training portion of each fold (TE-fold == model-fold), closing the

leakage path an in-sample profile mean would open. The leaderboard was **not submitted** (this was an unattended batch run), so all scores here are **CV/OOF-only**; no public/private LB score is recorded.

Models & Architecture

Purpose of Architecture. A regressor that minimizes RMSLE on media-campaign cost. **Architecture Type** is a **gradient-boosted decision-tree ensemble** — CatBoost as the primary learner (an Optuna-tuned member plus seed-bagged siblings), with a hand-set LightGBM member and a diversity deep-LGB member, combined by a Dirichlet-weighted blend; the champion is an **8-member blend** discovered at tree-search **node #22**.

Each member takes the same **Input Format** — a numeric feature matrix with the target modelled in `log1p` space — of **Input Dimension 21 features** per row (a 1-D vector; there is no spatial/sequence structure).

Architecture Description. Each GBDT member is an additive ensemble of decision trees grown by histogram-based gradient boosting. The representative Optuna-tuned CatBoost uses `depth = 10`, `learning_rate = 0.0824`, `l2_leaf_reg = 5.48`, `min_data_in_leaf = 33`, `random_strength = 0.092` (~300 boosting iterations after tuning vs ~1,300 for the hand-set original); the tree search additionally pushed CatBoost `depth` to **12**, beyond Optuna's own `depth ≤ 10` search cap, which became the single biggest lever. The blend layer sits on top as a convex weighted average of members' out-of-fold predictions, with weights found by **Dirichlet sampling**. **Model Complexity** is best expressed as trees × depth rather than a dense parameter count: the champion blends up to **8 members**, with weight concentrated in the depth-12 CatBoost seed family (~0.8 of total) plus a deliberately diverse deep-LGB member.

Training procedures

Training proceeds as a staged trajectory on the shared 5-fold split, each stage measured on the same OOF RMSLE scale:

Stage	Configuration	OOF RMSLE
1	generic batch baseline (15 raw features)	0.29723
2	skill base: <code>log1p</code> target + tuned params + early stopping (15 raw)	0.29710
3	skill engineered: + <code>amenity_count/ratios</code> + <code>store_te</code> K-fold TE (21 feats)	0.29614
4	Phase B: Optuna-tuned CatBoost + seed-bagging, 5-way blend (committed champion)	0.29565
5	tree-search v3 (<code>harness</code> , 24 nodes; node #22 = 8-member blend)	0.295284

The **Loss Function** is RMSE on the `log1p(cost)` target (LightGBM/CatBoost regression objective) — directly equal to the competition RMSLE — with predictions passed through `expm1` and clipped ≥ 0 as

a safety net. The **Optimization Algorithm** is gradient boosting (histogram, leaf-wise) — **not** SGD/ADAM — with blend weights optimized by Dirichlet sampling. The **Learning Rate** is the boosting shrinkage (tuned CatBoost `learning_rate` = 0.0824; hand-set LGB = 0.04); there is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by CV early-stopping, not a schedule) and **no Batch Size** (N/A — full-dataset histogram boosting, not mini-batched).

Training Duration for the tree-search run was **~513 s of wall-clock over 24 evaluated nodes** (12 solo / 12 blend, 0 failed); the Optuna tune was 40 trials / 319 s (fold-0 proxy), and a solo CatBoost fits in ~31–58 s (five prior pool solos were reused from cache at zero cost). **Training Memory** peak was **not recorded** — not explicitly capped, since a $360k \times 21$ float matrix is trivial on the single arm64 CPU machine. **Transfer Learning** is N/A (no pretrained weights); its analogue is **cross-competition experience injection** — the distilled "Optuna fold-proxy tune → pool → seed-bag" recipe (of which s3e11 is itself a validating comp, and the source of the "large data wants MORE capacity" reversal finding). **Data Augmentation** is N/A (tabular); the analogues are feature engineering (the 6 derived features) and seed-bagging.

Reproducibility Standards are strict: fixed seeds (fold seed = 42; Optuna and seed-bag seeds logged), CatBoost run with `allow_writing_files=False` and explicit `thread_count`, and per-round checkpointing (`npz`) so cached members are verified digit-for-digit against `experiments.json` rather than retrained. A **faithful replay reproduces the committed tree 24/24 exactly (max deviation $\Delta = +4e-6$** , pure npz-roundtrip float noise). Note: the tree in `experiments_tree_v3` was reconstructed from the v2 sweep's legacy OOF caches (`v3_native = false`).

Inference procedures

Decision Threshold is **N/A** — RMSLE is a regression metric, so we submit the raw predicted cost (`expm1` of the `log1p`-space prediction, clipped ≥ 0) and never threshold. **Inference Duration** was **not recorded** (not separately profiled); GBDT scoring of the 240,224 test rows is sub-second per member on CPU. **Inference Memory** is negligible (tree inference is lightweight, not capped).

Evaluation & Benchmarking

Task for Performance Evaluation. Predict media-campaign `cost` for every test row and minimize RMSLE. **Performance Metrics.** RMSLE is the sole competition metric; our committed Phase-B champion reaches **CV OOF RMSLE 0.29565**, improving $0.29723 \rightarrow 0.29710 \rightarrow 0.29614 \rightarrow 0.29565$ across the staged trajectory, and the tree-search v3 run drove the OOF mark further to **0.295284** (node #22, 8-way blend). Because this was an unattended batch run that was **never submitted to Kaggle**, no public/private LB score is recorded — all figures are CV/OOF-only.

Performance Benchmarking. The natural comparator is the **NVIDIA reproduce-agent**, which distills the essence of the strongest public kernel:

Agent	Approach	RMSLE	Note
AIDE	open-source AIDE agent	0.2941	local_winner
NVIDIA	reproduces essence of janmpia/feature-eng-xgb-cat-ensemble	0.2956	—
Our agent	from-scratch GBDT pool + tree-search	0.29565 (0.29528 at node #22)	—

Our number and NVIDIA's are **near-identical — a dead heat** (0.29565 vs 0.2956, a difference of $\sim 5e-5$ on a metric where fold RMSLE spans 0.2947–0.2976). AIDE, however, is ahead of both at 0.2941 and is the frozen table's `local_winner`. This is a genuinely low-signal dataset where every raw feature carries $|r| \leq 0.11$ with the target, so both a well-engineered from-scratch pipeline and a reproduced public ensemble converge to essentially the same floor. On this problem the reproduce-vs-originate comparison ends level, consistent with the broader sweep finding that the gap collapses on well-optimized modern Playground episodes.

Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08

All 20 baseline competitions were re-run in an isolated root (`~/benchruns/myagent-rerun`) under one frozen architecture (`docs/rerun_manifest.json`): every lane runs the **full v5 pipeline including the Stage 0.5 problem dossier** (dossier.json written in-lane before modeling), holds no credentials, and is checked by a per-lane transcript audit. Leaderboard scores for the re-run were produced by the separate credentialed operator scoring step and are recorded in `benchmark_results/rerun_three_way.csv` ; the figures below revise nothing in the sections above.

- **Lane:** done 2026-08-17, 107 min — first attempt (08-14) exited without a submission and was relaunched.
- re-run local CV: **RMSLE 0.292886** (5-fold on $\log_{10}(\text{cost})$), 10-member NNLS blend over tree-search solos + linear-stage honest pool.
- Public / private leaderboard for this re-run (`benchmark_results/rerun_three_way.csv`): **Public 0.29309 / Private 0.29368**; `winner_priv` = **mine** (AIDE 0.29445, NVIDIA 0.29624).